

The purpose of this assignment is to improve our understanding of MIPS assembly by creating a simple turn-based game with 4 objects; 1 player and 3 monsters with each having random health points and strength points. The game revolves around the player choosing either to attack strength amount or heal strength amount followed by the monsters each attacking the player with their strength amount with a 50% chance. The game ends when either the player's health points run out or there are no more monsters left to fight.

Here is how I approached the requirements in order:

First I researched how to generate a random number in MIPS and ended up creating the following label/procedure with 2 parameters.

a0: the minimum, a1: the maximum

```
49 randRange:
50     move $t0, $a0          #save min in $t0 to give the illusion that we can set a specific min in $a0 for simplicity, but its 0 unchangabl
51     sub $a1, $a1, $a0      # range_size = max - min subtract the min from our max because we can't set a min
52     addi $a1, $a1, 1       #make inclusive
53     li $v0, 42             # load random into v0
54     li $a0, 0              # RNG id = 0 $a0 original purpose is to set the rng ID, so we grab rng 0 numbers
55     syscall                # get the random int back in $a0
56     add $v0, $a0, $t0      # add minimum back to keep the illusion that we were able to set the minimum using $a0 in the function
57     jr $ra
58
```

\$a1 was the maximum on default and I saw people using \$a0 for the minimum but it wasn't until I tested it that I realized \$a0 was actually the seed for the random generation, and was preferably kept at 0. It made sense to use it as the minimum range so, in the label, I still pull the minimum value from \$a0, but I throw it into another register (\$t0) to deal with. The syscall 42 uses a1 as the max and uses 0 as the minimum, so to set the minimum number I had to subtract the minimum given to me from the maximum and then add that number back to create the illusion that I did have a custom minimum value set. The logic works like this:

\$a0 = 5

\$a1 = 15 # i need to create a random number between 5 - 15, but my min is set to 0

\$a1 = \$a1 - \$a0 # if i subtract the min from the max, I get my min to become 0

Having the range 0 - 10 to generate the random number using system call 42, and then add the min back to have the range 5 - 15 again.

That sums up my randRange that I consistently use in my program to generate random values

The next thing I focused on was creating the player. I created an array that can hold 4 objects of size 8 (4 for hp, 4 for st), so a size 32 array was created as gameData. The player occupies index 0, sitting at the very start of the array with health at offset 0 and strength at offset 4. I called randRange twice inside createPlayer, once for strength with a range of 5-12 and once for health with a range of 25-50, storing each result directly into gameData. Because createPlayer calls randRange using jal, I had to save \$ra on the stack before the calls and restore it after, otherwise the return address back to main would have been lost when jal overwrote it.

After creating the player I created the monsters in createMonster. The monsters occupy indices 1, 2, and 3 in the array, so their byte offsets are 8, 16, and 24. I looped through each monster index using \$s2 starting at 1 and going to 3, computing the byte offset by multiplying the index by the object size which I stored as a .word in memory under the label objectSize. A confusion I had here was trying to use objectSize directly in the mul instruction, but MIPS does not allow that because objectSize is a memory address not a register value. I had to first load it into a register using lw and then use that register in the mul.

<pre> 92 monsterLoop: # creation continued, simpl 93 94 bgt \$s2, 3, monsterLoopEnd # cor 95 96 # lw \$t2, objectSize 97 mul \$t0, \$s2, objectSize # \$t2 # </pre>	->	<pre> monsterLoop: # creation continued, bgt \$s2, 3, monsterLoopEnd lw \$t2, objectSize mul \$t0, \$s2, \$t2 # store </pre>
---	----	--

Each monster's health and strength were generated as a random number between one third of the player's corresponding stat as the minimum and 110% of the player's stat as the maximum, using integer division and multiplication to compute those bounds before each randRange call. A significant bug I ran into here was that after computing the monster's address and storing it, I was calling jal randRange which internally uses \$t registers. This corrupted my \$t registers which I was relying on to remember where in the array to store the result - my monster index. The fix was to store the monster's address in an \$s register, specifically \$s3, since \$s registers are callee saved and survive across jal calls. \$t registers are caller saved meaning any subroutine you call is allowed to overwrite them without warning.

```

92 monsterLoop: # creation continued, simplified, looped
93
94     bgt $s2, 3, monsterLoopEnd # condition to end loop and go to the next step, s2 can be 1, 2, 3
95
96     lw $t2, objectSize
97     mul $t0, $s2, $t2 # store the offset for each monster in $s3, because t1 will be corrupted in randRange
98
99     la $s3, gameData
100    add $s3, $s3, $t0
101    # ----- health -----
102    li $t2, 3
103    div $s0, $t2 # divide player health that we stored in $s0 with 3
104    mflo $a0 # load quotient into $a0 to use as min
105
106    li $t2, 110
107    mul $t3, $s0, $t2 # multiply by 110
108    li $t2, 100 # load 100
109    div $t3, $t2 # divide by 100 to get x110 /100 = 110%
110    mflo $a1 # store it in $a1 for max random
111
112    jal randRange # generate random value
113
114
115
116    sw $v0, 0($s3) # store random value as monster health first
117
118    # ----- strength -----
119    li $t2, 3
120    div $s1, $t2 # divide player strength that we stored in $s0 with 3
121    mflo $a0 # load quotient into $a0 to use as min
122
123    li $t2, 110
124    mul $t3, $s1, $t2 # multiply by 110
125    li $t2, 100 # load 100
126    div $t3, $t2 # divide by 100 to get x110 /100 = 110%
127    mflo $a1 # store it in $a1 for max random
128
129    jal randRange # generate random value
130    sw $v0, 4($s3) # store monster strength 4 from the health position
131
132    addi $s2, $s2, 1 # increment monster index
133    j monsterLoop # jump to start each monster gets a shot
134
135 monsterLoopEnd: # next step. clean up

```

For displayAll I looped through the monsters the same way, computing each offset and loading the health to check if it was greater than 0 before printing. Dead monsters are silently skipped. The player is always printed at the top. I originally had the display subroutine in multiple labels and subroutines, but I managed to simplify it into a couple subroutines and some helper labels :

```

141 displayAll:
142
143     # ----- print player -----
144     li $v0, 4
145     la $a0, str_player
146     syscall      # print "=== PLAYER ==="
147
148     li $v0, 4
149     la $a0, str_hp
150     syscall
151
152
153     li $v0, 1
154     lw $a0, gameData      # player health at offset 0
155     syscall
156
157
158     li $v0, 4
159     la $a0, str_str
160     syscall
161
162     li $v0, 1
163     lw $a0, gameData+4    # player strength at offset 4
164     syscall
165
166     li $v0, 4
167     la $a0, str_newline
168     syscall
169
170
171     # ----- print Monster loop
172
173     li $s0, 1 # monster index 1

```

```

174
175 displayLoop:

```

```

174
175 displayLoop:
176     bgt $s0, 3, displayDone    # if greater than the 3rd monster jump to done
177
178     lw $t2, objectSize
179     mul $t1, $s0, $t2    # multiply index x objectSize to get right offset
180     la $t0, gameData     # get array start
181     add $t0, $t0, $t1     # add offset to t0
182
183     lw $t2, 0($t0)        # load monster health
184     ble $t2, 0, skipMonster    # skip if dead
185
186     li $v0, 4
187     la $a0, str_monster    # print ---monster---
188     syscall
189
190     li $v0, 1
191     move $a0, $s0          # print monster index number
192     syscall
193
194     # --- Print Monster HP ---
195     li $v0, 4
196     la $a0, str_hp
197     syscall
198
199     li $v0, 1             # t0 is the monster health
200     lw $a0, 0($t0)
201     syscall
202
203     # --- Print Monster STR ---
204     li $v0, 4
205     la $a0, str_str
206     syscall
207
208     li $v0, 1
209     lw $a0, 4($t0)        # monster strength
210     syscall
211
212     li $v0, 4
213     la $a0, str_newline
214     syscall
215
216
217 skipMonster:
218     addi $s0, $s0, 1
219     j displayLoop
220
221 displayDone:
222     jr $ra
223

```

For playerTurn I handled three cases: attack, heal, and invalid input. Invalid input loops back to promptAgain, a label placed right after the stack setup, so the player is re-prompted without wasting their turn. This was the same looping back idea used in attackLoop where instead of a separate skip label I reused the loop's own entry point to retry with the next monster index. For healing I simply added the player's strength to the player's health and stored it back. For attacking I looped through monsters starting at index 1 and attacked the first one whose health was greater than 0 by subtracting the player's strength from it and storing the result back.

```

225 playerTurn:
226     # handling player input and action, no parameters needed
227     # it will use $s0 for player strength, $s1 for loop counter for finding first monster
228
229     addi $sp, $sp, -4
230     sw $ra, 0($sp)    # we are gonna call range which will use $ra, so lets save the return $ra
231
232 promptAgain:    # come back here to ask until he gives a 1 or a 2
233     li $v0, 4
234     la $a0, str_choice    # prompt the choices (1) attack, (2) heal
235     syscall
236
237     li $v0, 5    # get int input
238     syscall
239     move $t0, $v0    # move it to $t0
240
241     # invalid check
242     beq $t0, 1, doAttack    # if its 1 go do Attack
243     beq $t0, 2, doHeal    # if its 2 go do heal
244     li $v0, 4
245     la $a0, str_invalid    # prompt invalid if it gets here
246     syscall
247     j promptAgain    # loop back to ask the choices again
248
249 doHeal:
250     lw $t0, gameData    # load player health to $t0, we stopped using it
251     lw $t1, gameData+4    # load player strength into $t1
252     add $t0, $t0, $t1    # add heal = heal + strength
253     sw $t0, gameData    # save it back in our array
254     li $v0, 4    # print
255     la $a0, str_heal    # healed
256     syscall
257     j playerTurnDone    #jump to done
258
259
260 doAttack:
261     lw $s0, gameData+4    # load player strength to s0
262     li $s1, 1    # index the monsters
263
264 attackLoop:
265
266     lw $t2, objectSize
267     mul $t0, $s1, $t2    # stores the offset we will use
268
269     la $t1, gameData    # get the array to t1
270     add $t1, $t1, $t0    # add in the offset
271     lw $t2, 0($t1)    # load whats in that offset (index - moster health) into t2
272     bgt $t2, 0, attackHit    # monster has more health than 0, take the attck
273     addi $s1, $s1, 1    # that monster is dead, add an index value to monster index
274     bgt $s1, 3, playerTurnDone    # if index value crossed 3 just finish the turn, end the stuff
275     j attackLoop    # if we get all the way down here, that means we picked a dead monster, so we start over with
276
277 attackHit:
278     sub $t2, $t2, $s0    # t2 the monsters health value - s0 the players strength loaded at doAttack
279     sw $t2, 0($t1)    # store the resulting health back into t1 the address with the offset of the monsters h
280     li $v0, 4    # print
281     la $a0, str_attack    # the attack
282     syscall
283
284 playerTurnDone:    # returning
285     lw $ra, 0($sp)
286     addi $sp, $sp, 4
287     jr $ra

```

For the monsterTurn after player's, I looped through the monsters, each living monster rolled a random number between 1 and 10 using randRange, and if the result was above 5 the monster attacked the player by subtracting its strength from the player's health, this met the goal of creating a random set of numbers and picking half to represent attack and half to represent skip decision making. The same \$s3 fix applied here since registers \$t0-1 I used to have would have been corrupted by the jal randRange call before I could use it (for indexing) to read the monster's strength. By storing the monster address in \$s3 before the jal it remained valid after the call. I could've stored it in a bigger \$t register too, but this way it's safer.

```

289 monsterTurn:
290     addi $sp, $sp, -4
291     sw $ra, 0($sp)
292
293     li $s0, 1      # create its own indexing, start at index 1
294
295 monsterLoop1:
296
297     bgt $s0, 3, monsterTurnDone # if monsters 1-3 are done with, end it
298
299     lw $t2, objectSize
300     mul $t0, $s0, $t2 # create offset tracking on t0 based off of indexing at s0
301     la $s3, gameData # hold array start at s3, because the t0,t1 will get corrupted in randRange
302     add $s3, $s3, $t0 # combine the offset with the gameData address to get monster hp
303     lw $t2, 0($s3)    # load that monsters health
304     ble $t2, 0, monsterNext # if its zero skip monster and start over
305
306     # ----- 50% attack chance -----
307
308     li $a0, 1
309     li $a1, 10        # pick from a list of numbers 1-10 just like the assignment wants
310     jal randRange     # half of the numbers picked will be attack, half of the numbers picked will be skip
311
312     ble $v0, 5, monsterNext # if 5 or less skip turn for this monster, move onto what the next monster will do
313
314     # monster attacks
315
316     lw $t3, 4($s3)      # store monster strength
317     lw $t4, gameData    # store player health to t4
318     sub $t4, $t4, $t3    # sub the monster st from player hp
319     sw $t4, gameData    # store new player hp back to array
320     li $v0, 4           #print
321     la $a0, str_monsterhit # monster hit us
322     syscall
323
324 monsterNext:
325     addi $s0, $s0, 1
326     j monsterLoop1
327
328 monsterTurnDone:
329     lw $ra, 0($sp)
330     addi $sp, $sp, 4
331     jr $ra

```

For checkGameOver I first checked if the player's health was 0 or below, and if so printed the lose message and returned 1. Then I checked each monster's health one by one and if any was above 0 the game was still ongoing so I returned 0. If all three were 0 or below I printed the win message and returned 1. The return value of 1 or 0 in \$v0 was used in the main game loop to decide whether to exit or keep looping.

```
334 checkGameOver:
335
336     # at the end , lets check for win/lose conditions
337
338     lw $t0, gameData
339     ble $t0, 0, playerDead
340
341     # check all monster health to see if we can continue
342     lw $t0, gameData+8      # monster 1 health
343     bgt $t0, 0, gameOngoing # still alive, game continues
344     lw $t0, gameData+16     # monster 2 health
345     bgt $t0, 0, gameOngoing
346     lw $t0, gameData+24     # monster 3 health
347     bgt $t0, 0, gameOngoing
348
349
350     # if all monsters are dead
351
352     li $v0, 4
353     la $a0, str_win
354     syscall
355     li $v0, 1
356     jr $ra
357
358 playerDead:
359     li $v0, 4
360     la $a0, str_lose
361     syscall
362     li $v0, 1 # will indicate a stop
363     jr $ra
364
365
366 gameOngoing:
367     li $v0, 0 # will indicate a continue
368     jr $ra
369
```

The main game loop ran in this order each iteration: display, player turn, check game over, monster turn, check game over, then loop back. Checking after the player's turn meant that if the player killed the last monster the game ended immediately without giving the monsters a final retaliation turn, which was another issue I was facing.

```
24  .text
25      jal createPlayer
26      jal createMonster
27
28  gameLoop:
29      jal displayAll
30      jal playerTurn
31      jal checkGameOver
32      beq $v0, 1, gameExit
33      jal monsterTurn
34      jal checkGameOver
35      beq $v0, 1, gameExit
36
37      j gameLoop
38
39      #---- Exit ----
40  gameExit:
41      li $v0, 10
42      syscall
43
44
```

Here are the run-time screenshots of the work explained above:

Wars Messages	Run I/O
<pre> ==== PLAYER ==== HP: 45 STR: 8 ==== Monster 1 HP: 43 STR: 6 ==== Monster 2 HP: 19 STR: 6 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 1 You attacked the monster! A monster attacked you! A monster attacked you! ==== PLAYER ==== HP: 32 STR: 8 ==== Monster 1 HP: 35 STR: 6 ==== Monster 2 HP: 19 STR: 6 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 1 You attacked the monster! ==== PLAYER ==== HP: 32 STR: 8 ==== Monster 1 HP: 27 STR: 6 ==== Monster 2 HP: 19 STR: 6 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 1 You attacked the monster! ==== PLAYER ==== HP: 32 STR: 8 ==== Monster 1 HP: 19 STR: 6 ==== Monster 2 HP: 19 STR: 6 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 1 You attacked the monster! A monster attacked you! ==== PLAYER ==== HP: 25 STR: 8 ==== Monster 1 HP: 11 STR: 6 ==== Monster 2 HP: 19 STR: 6 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 1 </pre>	<pre> ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 1 You attacked the monster! A monster attacked you! A monster attacked you! A monster attacked you! ==== PLAYER ==== HP: 6 STR: 8 ==== Monster 1 HP: 3 STR: 6 ==== Monster 2 HP: 19 STR: 6 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 2 You healed yourself! A monster attacked you! ==== PLAYER ==== HP: 8 STR: 8 ==== Monster 1 HP: 3 STR: 6 ==== Monster 2 HP: 19 STR: 6 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 2 You healed yourself! A monster attacked you! A monster attacked you! ==== PLAYER ==== HP: 3 STR: 8 ==== Monster 1 HP: 3 STR: 6 ==== Monster 2 HP: 19 STR: 6 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 2 You healed yourself! Attack (1) or Heal (2): 2 You healed yourself! ==== PLAYER ==== HP: 11 STR: 8 ==== Monster 1 HP: 3 STR: 6 ==== Monster 2 HP: 19 STR: 6 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 2 You healed yourself! ==== PLAYER ==== </pre>

After years of healing and attacking:

Messages	Run I/O
	<pre> Attack (1) or Heal (2): 1 You attacked the monster! ==== PLAYER ==== HP: 22 STR: 8 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 2 You healed yourself! ==== PLAYER ==== HP: 30 STR: 8 ==== Monster 3 HP: 35 STR: 7 Attack (1) or Heal (2): 1 You attacked the monster! ==== PLAYER ==== HP: 30 STR: 8 ==== Monster 3 HP: 27 STR: 7 Attack (1) or Heal (2): 1 You attacked the monster! ==== PLAYER ==== HP: 30 STR: 8 ==== Monster 3 HP: 19 STR: 7 Attack (1) or Heal (2): 1 You attacked the monster! A monster attacked you! ==== PLAYER ==== HP: 23 STR: 8 ==== Monster 3 HP: 11 STR: 7 Attack (1) or Heal (2): 1 You attacked the monster! A monster attacked you! ==== PLAYER ==== HP: 16 STR: 8 ==== Monster 3 HP: 3 STR: 7 Attack (1) or Heal (2): 1 You attacked the monster! You win! </pre>

At runtime, the screenshots above show a player followed by 3 monsters being created with random health and strength values. The player is prompted with two choices: (1) Attack or (2) Heal. The player chooses to attack and their strength is subtracted from the first living monster's health. This continues until the player's health gets low enough that they start healing with (2), where their strength is added to their health. After repeating this loop for several turns, the player exhausts all the monsters and receives a win.

The biggest struggle throughout this assignment was understanding which registers survive a `jal` call and which don't. I kept losing values I needed because `randRange` was silently overwriting my `$t` registers mid-subroutine, and early on I was also losing `$ra` by forgetting to save it on the stack before calling `jal`. Once I understood that `$t` registers are not safe across calls and `$s` registers are, a lot of the bugs stopped happening. Overall this assignment was frustrating at times but it clicked once I stopped thinking in high level terms and started thinking about exactly what each register holds at every single step.